HPL · PLAYABLE ADS

Playable Editor

A Figma-style visual studio that turns imported designs into interactive, ad-network-ready playable ads — exported as a single reusable, self-contained file.

~10×
faster per MIP
(4 h → ~22 min)

23
built-in reusable game mechanics

8
ad networks from one export

<5 MB
single self-contained HTML output

- Figma import & paste
- Reusable game templates
- Cross-MIP project sync
- Code reshare via link
- Team library (cloud)
- Built-in QA
- One-click multi-network export

OVERVIEW

What the Playable Editor is

The Playable Editor is a purpose-built studio for coders and designers who ship **playable ads** — the interactive, mini-game creatives that run inside apps on networks like AppLovin, Mintegral, Unity, and Vungle. It lets a builder start from a **Figma import** or raw **asset import**, lay everything out on a **canvas that behaves exactly like Figma**, wire in behavior and mini-game mechanics, and export **one buildable, reusable, self-contained file** that runs on every network.

The problem it solves

A playable ad — internally, a **MIP** (marketing iteration / playable variant) — is normally hand-coded per client and per network. Even with AI assistance, one MIP takes **3–5 hours**: rebuilding the mockup, wiring interactivity, chasing WebView rendering quirks, and re-exporting for each network. The team's job is to **faithfully implement given mockups** — fast, accurate implementation is the whole value.

What it changes

With the editor, the same MIP takes **15–30 minutes**. Designs are **pasted or imported** rather than rebuilt; interactivity is **added, not coded**; mechanics are **reusable templates**; and export produces every network variant at once with the rendering fixes already baked in. The output is a single reusable file that can be re-opened, re-skinned, and shared.

The end-to-end workflow

Stage	What happens
1 · Import	Bring the design in from a **Figma URL**, **paste straight from Figma** (Copy as PNG/SVG → ⌘V), drop in **image/video/audio assets**, generate from a logo, or **recover an editable project from an already-built playable**.
2 · Edit	Arrange on a **Figma-style multi-frame canvas** (pan/zoom, snap guides, layers, alignment). Every scene is a live, isolated preview. Autosave + undo/redo throughout.
3 · Add behavior	Tag elements as draggable, drop-slots, scratch covers, tap-to-select, reveals; drop in one of **23 reusable mini-game mechanics**; add editable hand-guides, timers, CTAs, and animations.
4 · Review	Built-in **QA** (consistency across a client's MIPs, single-MIP engagement lint, version history) plus **per-network preflight** gates before you ship.
5 · Export	One click produces a **single <5 MB HTML file per network** (8 networks), with MRAID/ExitAPI, media compression, and AppLovin rendering fixes applied automatically.
6 · Share / Publish	**Reshare by short code/link**, publish to the **cloud team library**, or push to the AppLovin upload portal — all from inside the app.

> **The core promise:** import a finished design, add behavior instead of rebuilding it, and get one reusable file that runs everywhere — turning a half-day, AI-assisted coding job into a 15–30 minute assembly job.

Three-tier architecture (why the output is portable & safe)

runtime/

A **dependency-free DOM/CSS ad engine**. Reads a project + assets and renders/plays the ad. Ships as one standalone IIFE inlined into every export. Never imports editor code.

src/

The **React authoring app**. Renders each scene in a sandboxed `<iframe>` and talks to it only through a typed message protocol — **editor CSS can never leak into the ad**.

electron/

The **desktop shell**: native save/load, **ffmpeg media transcode**, and browser automation for portal uploads — reached through a minimal secured bridge.

FEATURES · 1 OF 4

Import anything, edit like Figma

Import & create methods

Figma URL import

Pull frames directly from a Figma file (token stored locally). A **detect** → **review** → **generate** flow with a progress bar; can skip image rendering (text/layout only) to dodge Figma's render-quota rate limits.

Paste straight from Figma

Figma "Copy as PNG/SVG" then `⌘/Ctrl+V` drops the design in — full-screen aspect becomes a scene **background**, smaller pastes become placed **images**. No token, API, or rate limit. The preferred "paste the design, then add behavior" path.

Asset import

Drag in **images, video, and audio**. Assets are stored in IndexedDB (not localStorage), compressed at export, and de-duplicated so media is written once per session.

Generate a MIP from scratch

For when there is no mockup: upload a logo + product, pick a game type, background style, and end-card options; a palette is extracted from the logo and **editable coded SVG** assets (not raster AI images) are produced as native, editable scenes.

Quiz-funnel generator

Paste quiz/survey text (blank line between questions, `*` marks the correct answer) and it generates a full intro → question scenes → end-card funnel — a format that otherwise takes ~8 hours by hand.

Round-trip import (recover from a build)

Open a previously **built** playable (single `.html`, a zipped network export, or a source zip) and recover a **fully editable project** — every export embeds its own project + assets, so builds are reversible.

The canvas — a Figma-style multi-frame world

Every scene renders as its own **frame** (a live, isolated iframe) on an infinite **pan/zoom world** over a dot grid. The active frame carries the full editing overlay: **selection, resize handles, pink snap guides, marquee select, inline text editing, and live dimension badges** that counter-scale to zoom.

Layout tools match a pro design tool: **6-way alignment, group / ungroup, bring-to-front / send-to-back**, a one-level **Layers** tree with drag-reorder and group folders, a resizable **Navigator** (scenes + layers), a context toolbar, and a status bar. Frame positions, zoom, and panel widths persist.

Editable element types — 12 primitives, all responsive by construction

background · bar (header/progress) · image · text (i18n) · CTA button · choice (quiz option) · hand-guide · countdown · dim / overlay · game-mount · endscene · unboxing

Each element carries geometry, alignment, animations (full preset library), layer blur, an **outside stroke** (radius + padding + fill), and can be turned into a **shape-mask container** (any silhouette — heart, star, etc.) holding an image *or* autoplay video. Text elements carry all locales in one file (browser-detected at runtime).

> **Isolation guarantee:** the ad always renders inside a sandboxed iframe and communicates through a typed protocol. The editor's theme styles the *tool*, never the *ad* — so what you preview is exactly what ships.

FEATURES · 2 OF 4

Reusable, editable game mechanics

Interactivity is a library, not a rebuild. Drop in a mechanic, feed it your assets, and it becomes an editable native scene. Mechanics are versioned plugins — one file + one registry line — so the whole team's library grows through normal code review.

23 built-in game templates

Match & memory

Find-match (flip pairs) · Match (drag to slot) · Sort · Merge (combine to upgrade) · Bingo (complete a line)

Reveal & luck

Scratch card · Scratch grid · Spin wheel · Spin-catch · Slots · Pick-a-box · Vending

Action & timing

Drag-to-target · Catch (drag basket) · Tap/pop bubbles · Slider · Swipe · Whack · Conveyor belt

Build & theme

Recipe (tap ingredients) · Word · Song-mix (model + song → video) · Embedded playable (HTML)

Each template exposes tunable parameters (grid size, counts, speeds, thresholds, colours) and named **asset slots** you fill with your own art. A new game auto-becomes a **starter template** for the whole team.

Element-level mechanics — behavior you tag onto any element, no game needed

Scratch-to-reveal (per element)

Mark any element a **scratch cover** and others as **money reveals**; scratching erodes the cover, floats a `$X.XX` popup, plays a reveal SFX, and **tallies into a header text** — one interactive scene replaces ~6 hand-faked reveal screens.

Drag & drop-slots

Tag elements **draggable** or a **drop-slot** (grouped, key-matched). Items follow the pointer, snap into matching slots, and completing a group advances the win scene.

Select → generate → reveal

Free-form **pick / fill / generate** tags: tap thumbnails to select N items across any categories you invent, then a circular %-progress "generates" and reveals a result image or video. Fully placed & styled by you.

Editable hand-guides

Every game seeds a **tailored route hint** (an editable multi-node hand path). Hands hide on first tap and reappear after idle. Fully editable: move nodes, swap the hand image.

Reuse across MIPs

Save as template & starters

Save any project as a reusable template; start a new MIP from a template or from any registered game. Template usage is tracked (which client/MIP used which mechanic).

MIP variants in one project

Same MIP, slightly different mechanics/win-conditions = **override layers** in one project. Variants override element properties only; export emits **one playable per variant**. Languages are handled separately (one file carries all locales).

FEATURES · 3 OF 4

Project sync, sharing & team collaboration

Cross-MIP projects & "Sync to project"

A real project layer above MIPs

Group related MIPs under a named **project** ("brand + date + theme"). The Home screen buckets cards by project; an in-editor switcher jumps between sibling MIPs and spins up "New MIP in this project."

Shared, live elements

Mark an element **Sync to project** and it becomes a true shared instance — edit its position, size, text, asset, colour, or animation in one MIP and it propagates to **every MIP** in the project. Per-element scope: **every scene** (watermark/overlay) or **one scene** per MIP.

Synced elements are ordinary elements carrying a sync key, so the runtime and export need **zero changes**. Shared media lives in IndexedDB; a registry reconciles every MIP on open.

Code reshare & reuse

Share by short code / link

Send any MIP as an **8-character code or link**. The recipient enters the code and gets a clean, standalone editable project (the sharer's project + sync markers are stripped). A one-shot snapshot, delivered through the cloud store — no file wrangling.

Cloud team library (Supabase)

An **opt-in** sync layer over the local library (still works fully offline). **Publish / pull** MIPs, set status, reassign owner, and browse the team library as a Home tab. Magic-link sign-in; role-based access.

QA, review & versioning

Consistency check

Flags style / SFX / animation inconsistencies **across a client's MIPs**; deep-links to the offending scene/element.

Engagement lint

Single-MIP checks: no interaction, dead-end scene, no CTA, tiny tap target, tiny text, broken asset ref, dead SFX.

Version history

Snapshot the design, label it, **diff any two versions**, and restore — all without touching current assets.

Results import (CSV) with a name-matched leaderboard is also built in for playtest data.

Desktop power (Electron)

Native file save/load · **ffmpeg video/audio transcode** at export (per-asset compression profiles matching the AppLovin recipe) · **browser automation** for the AppLovin upload portal · hardened networking (SSRF-guarded remote fetch, navigation locks).

FEATURES · 4 OF 4

One export, every network — production-hardened

Single reusable file, eight networks

Export assembles **one self-contained HTML file per network** — the dependency-free runtime is inlined, and the project + all assets are embedded as base64 (which is also what makes builds reversible back into editable projects). Per-network transforms (MRAID / ExitAPI / Vungle flag / zip wrapping) are applied automatically, and the **5 MB budget is enforced** with warnings for up-scaled/blurry assets.

AppLovin · ironSource · Unity · Google · Facebook · Mintegral · Vungle · Moloco

What makes the output reliable

Baked-in AppLovin WebView fixes

The runtime carries hard-won fixes for AppLovin's Chromium WebView: edge-gap bleed on bars, overlay dim edges, white-flash on overlay, immune-element z-index, and the endscene colour-leak. These recur on *every* hand-built MIP; here they ship for free.

Responsive by construction

A shared FIT/EXTEND model re-lays out every element and game on resize, zoom, and orientation. A 2026 audit confirmed **all 12 element types and 23/24 games** re-layout correctly, including floating win/lose overlays.

Cross-device sound

iOS audio priming (unlock all sounds on first gesture), looping gesture SFX, and per-event bindings mean win/reveal/scene sounds actually play on iPhone WebViews.

Preflight gate

Per-network checks (size limit, external requests, MRAID present, real click URL, has interaction + CTA) block a bad export before it leaves the building.

Preview & verification

A device-framed **preview overlay** (AppLovin, Phone, iPhone SE, Tablet — all editable) plays the real runtime, and a headless **smoke test** renders the actual exported HTML to confirm it mounts with no errors. CI runs typecheck → lint → build → test → smoke on every change; the codebase carries **20 test suites**.

> **Reusable by design:** one export is (a) a working ad on 8 networks, (b) re-openable back into a full editable project, (c) shareable by code, and (d) publishable to the team library — the same file, four ways.

IMPACT · TIME & COST ANALYSIS

What it saves: time & money

> **Key baseline note:** the **3–5 hours per MIP** figure is **already with AI assistance** (AI-assisted coding/generation). The editor reaches 15–30 minutes *while also reducing reliance on paid AI tooling* — so it cuts a **second** cost line (AI tokens/subscriptions) on top of labor. See the AI-cost note below.

Working assumptions

Assumption	Value	Basis
Time per MIP — current (AI-assisted, manual)	**4.0 h**	midpoint of 3–5 h
Time per MIP — with the editor	**0.375 h**	midpoint of 15–30 min
Working day (office hours)	**7.75 h**	7 h 45 m (given)
Current output baseline	**3 MIPs / day**	per designer (given)
Working days / year	**240**	~48 weeks × 5
Loaded labor cost (base case)	**$45 / h**	assumption; sensitivity shown

Per-MIP savings — the headline

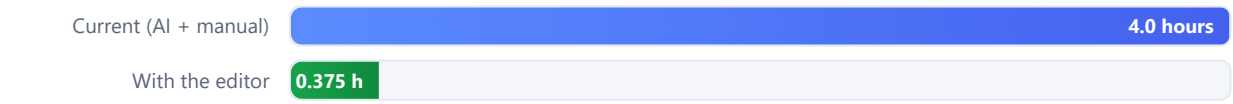
3.63 h
saved per MIP
(4.0 → 0.375 h)

91%
less time
per MIP

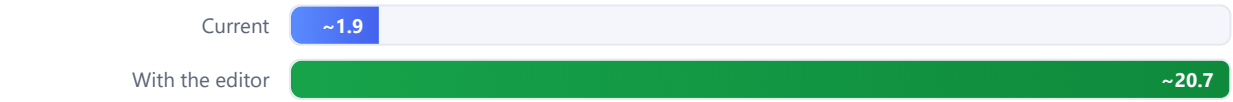
10.7×
faster
throughput

$163
labor saved / MIP
@ $45/h

Time per MIP



MIPs a single designer can finish in one 7.75-hour day



Two sides of the same coin: hold output flat and reclaim hours, or hold hours flat and multiply output ~10.7×.

IMPACT · THE NUMBERS

Daily, annual & team-level value

Producing the current 3-MIPs/day output

Per designer	Current (AI + manual)	With editor	Saved
Labor to make 3 MIPs	12.0 h	1.13 h	**10.9 h**
Labor cost / day @ $45/h	$540	$51	**$489**
Cost saved / day	—	—	**$489**

*Note the tension the numbers expose: 3 MIPs at 4 h each is **12 h** of work — more than a 7.75 h day. Hitting "3 MIPs/day" the manual way already needs overtime or ~1.55 designers. The editor makes 3 MIPs a **~1.1-hour** task for one person.*

Annual value (per designer, 720 MIPs/yr at the 3/day baseline)

Metric	Current	With editor	Saved
MIPs / year	720	720	—
Labor hours / year	2,880 h	270 h	**2,610 h**
Equivalent labor-days saved	—	—	**~337 days**
Labor cost saved / yr @ $45/h	**$129,600**	**$12,150**	**$117,450**

Sensitivity — labor cost saved per designer / year

Manual time / MIP ↓ Rate →	$30/h	$45/h	$60/h
3 h (best case for manual)	$56,700	$85,050	$113,400
4 h (base case)	$78,300	**$117,450**	$156,600
5 h (complex MIPs)	$99,900	$149,850	$199,800

Each cell = (manual hours − editor hours) for 720 MIPs/yr × rate. Editor time held at 0.375 h/MIP (270 h/yr).

At team scale (base case, $45/h)

$489
saved per designer per day

$117k
saved per designer per year

$587k
saved per year (team of 5)

13,050 h
labor hours freed / yr (team of 5)

Second saving — AI tooling spend. Because the 3–5 h baseline is **already AI-assisted**, most of the per-MIP AI generation/coding calls disappear: the editor replaces "AI writes/rebuilds it" with "paste + reuse a template." If AI tooling costs even a conservative **$3–$8 per MIP** today, that is another **~$2.2k–$5.8k per designer per year** (720 MIPs) in subscription/token spend removed — on top of the labor savings above, and it lowers vendor/API dependency risk.

All figures are modeled estimates from the stated assumptions (not measured), intended for planning. Labor rate, working days, and AI cost per MIP are the levers to localize to your team's actuals.

IN SUMMARY

Why it matters

For the builder

- Paste a Figma design instead of rebuilding it
- Add behavior by tagging, not coding
- 23 reusable, editable game mechanics on tap
- One click → every network, fixes included
- Re-open, re-skin, and reshare any past build

For the team & business

- ~10× faster per MIP; ~91% less build time
- ~$117k/designer/yr labor saved (base case)
- Lower AI tooling spend & vendor dependency
- Consistency & engagement QA before shipping
- Cloud library, code-share, project-level sync

The one-line pitch

> A Figma-style studio that turns imported designs into interactive playable ads and exports **one reusable file that runs on every network** — collapsing a **3–5 hour, AI-assisted coding job into 15–30 minutes**, and cutting both labor and AI-tooling cost in the process.

Feature checklist

✓ Figma URL import & paste-from-Figma
✓ Image / video / audio asset import
✓ Generate-a-MIP & quiz-funnel generators
✓ Recover editable project from a built file
✓ Figma-style multi-frame pan/zoom canvas
✓ 12 responsive element types
✓ Shape-mask containers (image or video)
✓ Layers, groups, align, snap guides
✓ 23 reusable, editable game mechanics
✓ Element-level scratch / drag / pick-generate
✓ Editable hand-guide route hints
✓ MIP variants + built-in i18n (all locales, one file)
✓ Cross-MIP projects + live "Sync to project"
✓ Share / import by short code & link
✓ Cloud team library (publish / pull, roles)
✓ QA: consistency, engagement, version diff
✓ Per-network preflight gate
✓ 8-network single-file export < 5 MB
✓ MRAID / ExitAPI, media compression (ffmpeg)
✓ AppLovin WebView rendering fixes baked in
✓ iOS audio priming & responsive re-layout
✓ Desktop app + AppLovin upload automation